

רמזים למטלת הכיתה החמישית

שבוע: 5 זמן משוער: פתח רק כשנתקעת

איך משתמשים בקובץ הזה

לכל קושי יש שלוש רמות:

רמז — כיוון. אסטרטגיה — איך לגשת. כמעט-פתרון — כמעט כל הקוד.

פתח רמה אחת. נסה **חמש דקות אמיתיות**. רק אז פתח את הבאה. אין ציון על מספר הרמות שפתחת, אבל יש הבדל עצום בין מי שנתקע ופתר לבין מי שהעתיק.

ולפני הכול: **פתח את `live-code/*/end/` מהשיעור** — כל קושי כאן כבר קרה לך פעם אחת במחזור, והקוד שהקלדת אז פתוח לידך. ו- `demos/02-tracks/` הוא כלי עבודה, לא תצוגה: תדביק בו כל רשימת מסלולים שאתה לא בטוח בה ותראה מה הדפדפן עושה איתה.

א. "כתבתי `display: grid` ושום דבר לא זז"

רמז. מה כתבת אחרי זה? `display: grid` לבדו נותן לך `grid` של **עמודה אחת** — וזה נראה בדיוק כמו כלום, כי ערימה זה מה שהיה שם קודם.

אסטרטגיה. ל-`grid` צריך גם **מסלולים**. פתח את כלי הפיתוח, בחר את המכל, וחפש את התג הקטן `grid` ליד השם שלו — לחיצה עליו מציירת את הקווים על העמוד. אם אתה רואה קו אחד, יש לך עמודה אחת.

כמעט-פתרון.

```
.something {
  display: grid;
  /* without the next line you have one column, and that is not a layout */
  grid-template-columns: /* ... */;
  gap: /* ... */;
}
```

ב. "הציור יוצא נכון ברוחב אחד ומתעוות בכל השאר"

רמז. הסתכל מה כתבת במסלולים. אם יש שם `px`, בנית ציור בגודל אחד.

אסטרטגיה. המספרים 50, 153, 198, 320 הם **יחסים**, לא מידות. `fr` מקבל כל מספר — לא רק 1 — אז אפשר להשתמש בהם בדיוק כמו שהם ולקבל את אותו ציור בכל רוחב. ואת הריבועיות מקבלים מתכונה אחת, לא מגובה.

כמעט-פתרון.

```
.mondrian {
  display: grid;
  grid-template-columns: /* four numbers, in fr */;
  grid-template-rows: /* four numbers, in fr */;
  gap: 9px;

  width: min(100%, 34rem);
  aspect-ratio: /* ... */;
  background: var(--mondrian-line); /* the lines */
}
```

ג. "הקווים השחורים בציור לא מופיעים"

רמז. אתה מנסה לצייר אותם. אל תצייר אותם.

אסטרטגיה. ה-`gap` הוא **חור** בין התאים, ודרך חור רואים מה שמאחור. מה יש מאחורי התאים? המכל. תן למכל צבע רקע שחור, ותן לכל תשעת האזורים רקע משלהם — והחורים ביניהם יהפכו לקווים בעצמם.

כמעט-פתרון. אין כאן `border` על אף אלמנט, ואין אלמנט לאף קו. שתי הצהרות רקע, וזהו.

ד. "האזורים בציור לא במקום, או שיש לי חורים לבנים"

רמז. ארבע עמודות וארבע שורות זה **שישה עשר תאים**, ויש לך רק תשעה אזורים. רובם מכסים יותר מתא אחד.

אסטרטגיה. מקם לפי **קווים**, לא לפי תאים. ארבע עמודות זה חמישה קווים, מ-1 עד 5. אותו דבר לשורות. הקיצור `grid-area` מקבל ארבעה מספרים בסדר הזה:

```
grid-area: row-start / column-start / row-end / column-end;
```

הסתכל ב- `images/warmups.png` וסמן על נייר איפה כל אזור מתחיל ואיפה נגמר. **צייר את זה לפני שאתה מקליד.**

כמעט-פתרון. האדום הוא התא השמאלי העליון בלבד — כלומר משורה 1 לשורה 2 ומעמודה 1 לעמודה 2:

```
.m-red {
  grid-area: 1 / 1 / 2 / 2;
}
```

הלבן שמימין לו באותה שורה תופס את שלוש העמודות הנותרות, כלומר מעמודה 2 עד הקו האחרון. משם תמשיך לבד. אם אתה משתמש ב- `-1` בשביל "הקו האחרון", זה עובד וזה גם עדיף.

ה. "בלוח יצאו לי פסים אנכיים במקום לוח שחמט"

רמז. זה בדיוק מה שאמור לקרות עם `:nth-child(odd)`, וטוב שראית את זה.

אסטרטגיה. ספור בקול: משבצת 1 בהירה, 2 כהה, ... 8 כהה, ואז 9 — צריכה להיות **כהה**, כי היא מתחילה שורה חדשה. אבל 9 הוא אי-זוגי, אז `odd` צובע אותה בהיר. הזוגיות לא מתהפכת בסוף השורה כי הרחב זוגי.

אז מה **כן** חוזר? מצא את **המחזור**: אחרי כמה משבצות התבנית חוזרת בדיוק להתחלה — לא בסוף שורה אחת, אלא כשגם הצבע של המשבצת הראשונה בשורה חוזר. זה המספר שלפני ה- n .

אסטרטגיה בשלושה צעדים — זו אותה שיטה שעשינו במחזור 2 על הרצפה ברוחב 4:

1. צייר על נייר את השורות הראשונות של הלוח שלך, ומספר כל משבצת.
2. ספור אחרי כמה משבצות הדפוס חוזר. ברצפה ברוחב 4 זה יצא $8n$; שאל את עצמך למה, ומה מקביל לזה ברוחב שלך.
3. בתוך מחזור אחד, רשום את מספרי המשבצות הבהירות. כל אחד מהם הוא $k +$ אחד. תן לכולן את הצבע הכהה, ואז החזר את הבהירות ברשימה של סלקטורים.

```
.board > * {
  aspect-ratio: 1;
  background: var(--board-dark);
}

/* one selector per light square inside a single period – you count them */
.board > :nth-child(/* period */ n + /* position */) {
  background: var(--board-light);
}
```

בדיקה בלי מורה: הסתכל על המשבצת הראשונה בשורה הראשונה ועל המשבצת הראשונה בשורה השנייה. בלוח הן בצבעים שונים; אם הן באותו צבע — עדיין יש לך פסים.

ו. "השלד לא מסתדר, או שאזור אחד קופץ למקום אחר"

רמז. האם כל חמשת האזורים תובעים שם? אזור בלי `grid-area` ממוקם אוטומטית ונוחת בתא הפנוי הראשון — שזה בדרך כלל לא איפה שרצית.

אסטרטגיה. עבוד בשני שלבים ואל תערבב ביניהם:

1. **המכל** מקבל את התמונה: `grid-template-areas`, שורה אחת בגרשיים לכל שורה בפריסה.
2. **כל אזור** מקבל `<שם> grid-area:` וזהו. שום מספר, שום קו.

שני כללים ש-Grid לא יסלח עליהם, ולא יגיד לך עליהם כלום:

- **בכל שורה חייב להיות אותו מספר תאים.**
- **כל אזור חייב להיות מלבן.** אין צורת L.

אם הפרת אחד מהם, הדפדפן **זורק את כל ההצהרה בשקט** ואתה מקבל פריסה אוטומטית שנראית מבולגנת בלי שנכתב שום דבר לקונסולה. `demos/05-placement/02-areas/` מראה את זה עם הודעות שגיאה אמיתיות — תדביק שם את התמונה שלך.

כמעט-פתרון.

```
.layout {
  display: grid;
  grid-template-areas:
    'masthead'
    'main'
    /* ... */;
  grid-template-columns: minmax(0, 1fr);
}

.site-header { grid-area: masthead; }
/* ... and four more ... */
```

ז. "התוכן הראשי מופיע אחרי התפריט בנייד ואני רוצה אותו לפני"

רמז. אתה לא צריך לגעת ב-HTML, ואסור לך.

אסטרטגיה. התמונה של האזורים קובעת את הסדר **הוויזואלי**, והיא לא חייבת להסכים עם סדר ה-DOM. פשוט כתוב את `main` לפני `side` בתמונה של הבסיס.

זה אחד המקומות הבודדים שבהם זה לגיטימי – התפריט הוא landmark ומי שמשתמש בקורא מסך מגיע אליו ישירות, כך שהסדר בקוד נשאר הנכון עבורו. באלמנטים אינטראקטיביים שצריך לעבור עליהם ב-Tab לפי סדר, זה כן היה באג.

ח. "הכרטיסים לא משנים מספר עמודות, או שנשארים מסלולים ריקים"

רמז. אם כתבת `repeat(3, 1fr)` – זה תמיד שלוש, גם בנייד. אתה צריך "כמה שנכנסים".

אסטרטגיה. שתי מילות המפתח הן `auto-fill` ו-`auto-fit`, והן **זהות** כשיש מספיק כרטיסים למלא את השורה. הן נפרדות רק כשיש **פחות פריטים ממסלולים**: `auto-fill` משאיר את המסלולים הריקים עומדים, `auto-fit` מקריס אותם לאפס והשורדים גדלים.

הרץ את `demos/04-auto-fit-fill/` וגרור את מספר הפריטים למטה. תראה את ההבדל בשלושים שניות.

והמסלול עצמו צריך **רצפה** – מתחת ל-15rem הכרטיסים האלה מפסיקים להיות קריאים.

כמעט-פתרון.

```
.kpis {
  display: grid;
  grid-template-columns: repeat(/* fit or fill */, minmax(/* the floor */, 1fr));
  gap: var(--space-4);
}
```

ט. "השורה האפורה בתחתית הכרטיסים לא מיושרת"

רמז. זו אותה בעיה מהשבוע שעבר, ואותו פתרון. הכרטיס הוא **רכיב**, לא עמוד.

אסטרטגיה. הכרטיס הוא עמודת Flexbox. משהו באמצע צריך לבלוע את המקום הפנוי, וכל מה שאחריו נדחף למטה. אם תהפוך את הכרטיס ל-Grid זה גם יעבוד — וזה יעלה לך נקודות באיכות הקוד, כי זה הכלי הלא נכון לציר אחד.

כמעט-פתרון.

```
.kpi {
  display: flex;
  flex-direction: column;
}

.kpi-note {
  flex: 1; /* swallows the space, and pushes what follows to the bottom */
}
```

י. "יש לי גלישה או פס גלילה אופקי ואני לא מוצא ממה"

רמז. חפש מסלול `1fr` שיש בתוכו משהו שלא נשבר.

אסטרטגיה. `1fr` הוא בעצם `minmax(auto, 1fr)`. המקסימום הוא חלק מהמקום הפנוי — זה מה שכולם קוראים — אבל **המינימום הוא `auto`**, כלומר לפחות רוחב התוכן. טבלה, כתובת ארוכה או מילה שלא נשברת ידחפו את המסלול והוא לא יתכווץ.

התיקון הוא `minmax(0, 1fr)`.

זה בדיוק הבאג של שבוע שעבר — `min-width: auto` על פריטי `flex`, ו-`min-width: 0` כתיקון. אותה רצפה, כתיב אחר. `demos/07-minmax-0/` מודד את זה.

והפקודה מהשבוע שעבר עדיין עובדת — הדבק בקונסול:

```
document.querySelectorAll('*').forEach((el) => {
  if (el.getBoundingClientRect().right > document.documentElement.clientWidth + 1)
    console.log(el);
});
```

יא. "הצרתי את החלון והכול נשבר / נקודות השבירה נלחמות זו בזו"

רמז. חפש `max-width` בקובץ. אם יש שם אחד, זו כנראה הסיבה.

אסטרטגיה. תכתוב מהקטן כלפי מעלה. **הבסיס — בלי שום שאילתה — הוא מה שנייד מקבל**, וכל שאילתה מעל זה רק **מוסיפה** את מה שהשתנה. ככה שתי שאילתות אף פעם לא סותרות זו את זו.

בדרך ההפוכה כל שאילתה צרה יותר חייבת **לבטל** את מה שהרחבה קבעה, ואת הביטול הזה שוכחים.

`demos/08-responsive/01-breakpoints/` מראה את שתי הגרסאות זו לצד זו ומדגיש איזה בלוק פעיל בכל רוחב.

ואם אתה תוהה איפה לשים את המספרים: **הצר עד שזה נשבר**. שם.

יב. "התמונות בלוח הסטטוס מעוכות, או שהשורה קופצת כשהן נטענות"

רמז. שתי בעיות שונות, ושתי תכונות שונות. אל תנסה לפתור את שתיהן באחת.

אסטרטגיה.

• **הקפיצה** קורית כי לא הוזמן מקום לתמונה מראש. `aspect-ratio` על ה-`img` מזמין את התיבה לפני שהקובץ הגיע.

• **העיוות** קורה כי התמונה בצורה אחרת מהתיבה. `object-fit` מחליט מה לעשות עם זה: `cover` ממלא וחותר, `contain` מקטין ומשאיר שוליים.

לצילומים `cover`. ללוגו או לתרשים שהקצה שלו נושא משמעות — `contain`.

יג. "סיימתי את הליבה. מה עכשיו?"

בסדר הזה:

1. **שכבת ההרחבה.** התמונות והטיפוגרפיה הזורמת הן עשרים נקודות, והן מהירות.
2. `grid-auto-flow: dense` ואז הכרטיס הרחב. שים לב שהוא צריך להיות רחב **רק** כשיש יותר ממסלול אחד.
3. `subgrid`. קרא את הסעיף בקובץ הסגנון, ואז את [ההסבר של MDN](#). עטוף ב-`@supports` — זה נדרש, וזה גם הרגל שיישאר איתך.
4. אם נשאר זמן: פתח את `demos/07-minmax-0/` בתיקיית השבוע, ואז את קוד המקור שלו (View Source). הוא מראה ש-`1fr` הוא בעצם `minmax(auto, 1fr)`, ולמה `minmax(0, 1fr)` ו-`min-width: 0` על הילד עוצרים את הגלישה. עכשיו אתה יודע בדיוק למה.
5. ולתרגול נוסף: [CSS Grid Garden](#) — אותו רעיון כמו Flexbox Froggy מהשבוע שעבר, ואפשר לסיים אותו בעשרים דקות.